# Homework 2 - Tracking *Java Security* weaknesses using *GitHub*

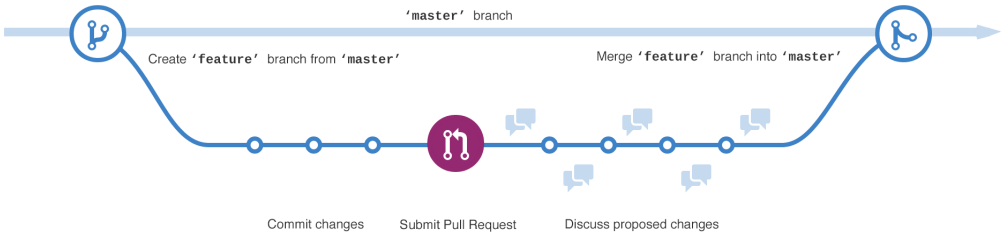
CIS4615 assigned 23 Sep 2025
Michael McAlpin, Instructor due 05 Oct 2025

## 1 Objective

Create a *GitHub* repository for **10** short Java programs with known security problems as identified in the *SEI CERT Oracle Coding Standard for Java*, then solve those security problems and use *GitHub* to track applied changes.

## 2 Requirements

*GitHub* will be the major part of this assignment and the next assignment. First off, here's a quick *GitHub* overview. *GitHub* is a code hosting platform for version control and collaboration. It lets you and others work together on projects from anywhere. *GitHub* uses the following elements and actions to manage codebases - for a single user or *many* users. Some important ideas are shown in the diagram below.




And are described as follows:

**Repository** A collection of one to many files.

**Branch** A subset of the *repository* consisting of a file or files.

**Pull** The act of selecting a file or files to put into a *branch* that will be updated or undergoing some modifications.

**Merge** Accepts all the modifications to the selected file or files, and updates the *Master* branch with those modifications. This merged product will be the new baseline *Master*.

It is important to note that this can be accomplished thru the *GitHub* website or by using the *GitHub* application installed on your PC. A very good command line tutorial is available at https://www.liquidlight.co.uk/blog/git-for-beginners-an-overview-and-basic-workflow/. (This is a very useful tutorial for those of you comfortable with the Linux & Mac OSX terminal.)

## 2.1 GitHub Account

A *GitHub* account is required for this assignment for two primary reasons. The first is to walk through a modern software production environment that uses some form of version control and change tracking. The second reason is that another assignment will require a *GitHub* account. We will be using this *GitHub* account for subsequent assignments.

Establishing a *GitHub* account for this assignment can be accomplished using the following steps:

1. Signup for a *GitHub* account https://github.com/
   (a) Select the "Free Plan".
   (b) Answer the profile questions. (I answered the account was for school work, hosting assignments, etc.)
   (c) Create a **public** repository for this class (I named it cis4615a. I strongly suggest you do likewise.)
      i. Read the "hello-world" guide page. https://guides.github.com/activities/hello-world/
      ii. Make sure you enable pulls from this repository
      iii. Create a "master" branch
   (d) Grab the program named **cis4615a.java** from the **MMcUCF** github repository.
      i. Download that file, edit it to include your name instead of the phrase " - from the class of CIS4615 - Fall 2022 - UCF". (That is it will be the string " - from YOUR NAME HERE". Really, your first & last name.)
      ii. Test it.
      iii. Upload the file to an "edits" branch.
         A. You'll need to create a branch from your "master" branch.
         B. This step is documented in the "hello-world" tutorial mentioned above.)
      iv. Note the code differences when viewing the source code file in the github "edits" branch you just created.
      v. Merge the "edits" branch with the "master" branch.
   (e) Congratulations - the create, pull, edit, and merge github process is now at your fingertips.

Note that you have created a *repository*, a *branch*, added some code, pulled from another *GitHub* account, modified the code, and merged it back into the *repository*. Those same steps will be applied to the following problems - *with one exception - the changed code will* ***not*** *be merged into the repository. More about that later.*

## 2.2 *SEI CERT Oracle Coding Standard for Java* problems

The Java rules and recommendations used in the *SEI CERT Oracle Coding Standard for Java* are a work in progress and reflect the current thinking of the secure coding community. Because this is a development website, many pages are incomplete or may contain errors. Nonetheless, we will be using this for a baseline set of a total of 10 short programming examples with *known security* deficiencies. There are **5 mandatory** examples to be included in your *GitHub* account and **5 user selected** examples to be included in your *GitHub* account to earn full credit for this assignment.

The *Coding Standard* consists of a set of *rules* which are defined as follows: Rules must meet the following criteria:

- Violation of the guideline is likely to result in a defect that may adversely affect the safety, reliability, or security of a system, for example, by introducing a security flaw that may result in an exploitable vulnerability.
- The guideline does not rely on source code annotations or assumptions.
- Conformance to the guideline can be determined through automated analysis (either static or dynamic), formal methods, or manual inspection techniques.

Rules are identified by the label *rule*, for example **Rule 00. Input Validation and Data Sanitization (IDS)** as shown below.

Each rule has one to several examples. The examples selected for this exercise are specified below.

### 2.2.1 5 Mandatory Vulnerabilities

1. **Rule 00. Input Validation and Data Sanitization (IDS)**
   *IDS03-J. Do not log unsanitized user input* Given the *non-compliant* code below:

```
if (loginSuccessful) {
  logger.severe("User login succeeded for: " + username);
} else {
  logger.severe("User login failed for: " + username);
}
```

   Correct the code as shown in the*Compliant Solution* below:

```
if (loginSuccessful) {
  logger.severe("User login succeeded for: " + sanitizeUser(username));
} else {
  logger.severe("User login failed for: " + sanitizeUser(username));
}
public String sanitizeUser(String username) {
  return Pattern.matches("[A-Za-z0-9_]+", username))
      ? username : "unauthorized user";
}
```

   Refer to: Rule 00. Input Validation and Data Sanitization (IDS)

2. **Rule 02. Expressions (EXP)**
*XP00-J. Do not ignore values returned by methods* Given the *non-compliant* code below:

```
public void deleteFile(){

  File someFile = new File("someFileName.txt");
  // Do something with someFile
  someFile.delete();
}
```

Correct the code as shown in the*Compliant Solution* below:

```
public void deleteFile(){

  File someFile = new File("someFileName.txt");
  // Do something with someFile
  if (!someFile.delete()) {
    // Handle failure to delete the file
  }

}
```

Refer to: Rule 02. Expressions (EXP)

3. **Rule 03. Numeric Types and Operations (NUM)**
*NUM03-J. Use integer types that can fully represent the possible range of unsigned data*
Given the *non-compliant* code below:

```
public static int getInteger(DataInputStream is) throws IOException {
  return is.readInt();
}
```

Correct the code as shown in the*Compliant Solution* below:

```
public static long getInteger(DataInputStream is) throws IOException {
  return is.readInt() & 0xFFFFFFFFL; // Mask with 32 one-bits
}
```

Refer to: Rule 03. Numeric Types and Operations (NUM)

4. **Rule 04. Characters and Strings (STR)**
*STR03-J. Do not encode noncharacter data as a string* Given the *non-compliant* code below:

```
BigInteger x = new BigInteger("530500452766");
byte[] byteArray = x.toByteArray();
String s = new String(byteArray);
byteArray = s.getBytes();
x = new BigInteger(byteArray);
```

Correct the code as shown in the*Compliant Solution* below:

```
BigInteger x = new BigInteger("530500452766");
String s = x.toString();  // Valid character data
byte[] byteArray = s.getBytes();
String ns = new String(byteArray);
x = new BigInteger(ns);
```

Refer to: Rule 04. Characters and Strings (STR)

5. **Rule 06. Methods (MET)**
*MET01-J. Never use assertions to validate method arguments* Given the *non-compliant* code below:

```
public static int getAbsAdd(int x, int y) {
  assert x != Integer.MIN_VALUE;
  assert y != Integer.MIN_VALUE;
  int absX = Math.abs(x);
  int absY = Math.abs(y);
  assert (absX <= Integer.MAX_VALUE - absY);
  return absX + absY;
}
//Usage:  getAbsAdd(Integer.MIN_VALUE, 1);
```

Correct the code as shown in the*Compliant Solution* below:

```
public static int getAbsAdd(int x, int y) {
  if (x == Integer.MIN_VALUE || y == Integer.MIN_VALUE) {
    throw new IllegalArgumentException();
  }
  int absX = Math.abs(x);
  int absY = Math.abs(y);
  if (absX > Integer.MAX_VALUE - absY) {
    throw new IllegalArgumentException();
  }
  return absX + absY;
}
```

Refer to: Rule 06. Methods (MET)

### 2.2.2 5 User selected Vulnerabilities

Based on the *5 items above* select 5 other well known Java vulnerabilities. Use the same procedure outlined above for the *mandatory* vulnerabilities. *Make sure to choose 5 different rule sets with a single example from the 5 unique rules.*

### 2.2.3 Filenames

| Rule | Filename |
|---|---|
| Rule 00. Input Validation and Data Sanitization (IDS) | R00_IDS03_J.java |
| Rule 02. Expressions (EXP) | R02_XP00_J.java |
| Rule 03. Numeric Types and Operations (NUM) | R03_NUM03_J.java |
| Rule 04. Characters and Strings (STR) | R04_STR03_J.java |
| Rule 06. Methods (MET) | R06_MET01_J.java |

Table 1: SEI Java Rules, examples and filenames.

The filenames shown above for the 5 *mandatory* code fragments are the concatenation of the *rule number* and the *specific code example* separated by an *underscore* (_). Use that same technique for name generation for the 5 examples you choose.

# 3 Submission - via WebCourses

Place the link for your assignment's *GitHub* account in a `DOCX`, `DOC`, or `TXT` file and submit the file via WebCourses.

# 4 Grading

Grading will be based on the following rubric:

| Deduction | Description |
|---|---|
| -100 | No account on *GitHub*. |
| -100 | No files on *GitHub*. |
| - 15 | No *Mandatory* or *User selected* vulnerability for the each **Rule** and **Example**. Deducted for each occurrence. |
| - 10 | Code fragment does not specify the **Rule** and **Example** as specified in the program's name defined above in *Table 1 SEI Java Rules, examples and filenames*. Deducted for each occurrence. |

Table 2: Grading Rubric